

FieldFlow

API Reference

FieldFlow API Reference

Base URL (production): `https://chatbot-492915.el.r.appspot.com`

Base URL (local dev): `http://localhost:8080`

All endpoints under `/api/`. The frontend SPA is served on every other path.

Authentication

Every authed request must send `Authorization: Bearer <firebase-id-token>`. The token comes from the Firebase Auth client SDK after a successful login.

Public endpoints (no auth):

- `POST /api/auth/sheet-login`
- `POST /api/auth/login`
- `POST /api/send-otp`
- `POST /api/verify-otp`
- `GET /api/wa/webhook` (Meta verification challenge)
- `POST /api/wa/webhook` (Meta inbound events — signature-verified instead)
- `POST /api/whatsapp/webhook` (legacy Glific webhook)
- `GET /api/whatsapp/health`
- `GET /api/_ah/warmup` (App Engine warmup)

Every other endpoint requires the Bearer token. `req.user` is populated with:

```
{
  uid:    string    // Firebase UID
  orgId:  string    // UUID of the user's organisation
  role:   'superadmin' | 'admin' | 'manager' | 'employee'
  name:   string
  phone:  string
}
```

Error format

All endpoints return JSON. Errors look like:

```
{ "error": "Human-readable message", "cid": "abc12345" }
```

The `cid` (correlation ID) appears in server logs — give it to support when reporting issues.

Rate limits

Scope	Limit
AI endpoints (Notebook chat, get-ai-report, etc.)	30 req/min/IP
/api/prewarm	10 req/min/IP
/api/auth/sheet-login, /api/verify-otp	8 req/min/IP
/api/send-otp	3 req/min/IP

Returns 429 Too Many Requests with a Retry-After header when exceeded.

1. Authentication

POST /api/auth/sheet-login [public]

Sheet-based credential login (DB + Google Sheet fallback).

Body: { phone, password, orgSlug } → Response: { customToken }

POST /api/auth/login [authed]

After Firebase phone-OTP auth, ties the UID to a DB user + sets custom claims.

Response: { success, orgId, metadata }

POST /api/send-otp [public]

Send OTP via Twilio / Fast2SMS / WhatsApp. Body: { phone, channel: 'sms' | 'whatsapp' }

POST /api/verify-otp [public]

Verify the OTP. Body: { phone, otp } → { valid }

2. Org & Subscription

GET /api/org/metadata [authed]

Current org's metadata (branding, modules, projects, custom_fields, ...). Returns: { orgId, metadata, subscription? }

PUT /api/org/metadata [admin]

Update org metadata. Body: { metadata: { ...partial fields... } }

PUT /api/org/content-hub-permissions [admin]

PUT /api/org/tab-permissions [admin]

PUT /api/org/user-access [admin]

GET /api/subscription [authed]

Plan + status for the current org.

3. Workers (field staff)

Method	Path	Role
POST	/api/add-user	admin
PUT	/api/update-user	admin
DELETE	/api/delete-user	admin
PATCH	/api/set-user-active	admin
POST	/api/merge-users	admin
POST	/api/analytics/worker	admin/manager

Add/update body: { name, phone, state, role, manager, password, employee_id, designation, project_ids[] }

4. Reports

POST /api/get-ai-report [authed] **(SSE)**

Generate a Field/Team/Project/Org/Impact/Story report via Gemini. Streams tokens.

Body: { reportType, filters, instruction, count?, language?, ... }

POST /api/drive-folder [authed]

Save the generated DOCX to Google Drive. Body: { docxBase64, filename }

POST /api/generate-social-post [authed] **(SSE)**

Body: { platform, tone, images, language, customPrompt }

POST /api/analyze-report-impact [authed]

Per-report Theory-of-Change + barriers analysis. Body: { report }

POST /api/analyze-toc-aggregate [authed]

Org-wide ToC across many reports. Body: { reports, scopeLabel }

GET /api/proxy-image [authed]

CORS-friendly image proxy for tags. Query: ?url=<https...>

POST /api/prewarm [authed]

Wake up Cloud Run notebook/report-writer instances.

5. Saved Reports

Method	Path	Purpose
POST	/api/saved-reports	Save a generated report
GET	/api/saved-reports	List saved reports for the org
GET	/api/saved-reports/:id	Fetch one
DELETE	/api/saved-reports/:id	Delete

6. Quick Report (AI extraction from free text)

POST /api/reports/quick-extract [authed]

Body: { text, projectsHint? } → { fields: { location, state, project, area_of_intervention, beneficiaries, description }, confidence, original_text }

POST /api/reports/quick-submit [authed]

Body: { fields, original_text, photo_urls? } → inserts a daily_reports row, returns { id, success }

7. Action Plan (multi-project)

Plan management

Method	Path	Role	Purpose
GET	/api/action-plans	manager+	List active plans for the org. <code>?include_archived=1</code> to include soft-deleted.
POST	/api/action-plans	manager+	Create plan from parsed XLSX. Body: { name, year, start_month, locations[], activities[] }
GET	/api/action-plans/:id	manager+	Fetch full plan including activities JSON
DELETE	/api/action-plans/:id	manager+	Soft-delete (set <code>active=false</code>). <code>?purge=1</code> for hard-delete with cell data
POST	/api/action-plans/:id/restore	manager+	Undo soft-delete
GET	/api/action-plans/archived	manager+	List soft-deleted plans
GET	/api/action-plans/template.xlsx	manager+	Download blank Excel template
GET	/api/action-plans/impact	manager+	Org-wide aggregation across all active plans (used by Impact Dashboard)

Cell data (target / achieved / notes)

All accept optional `?plan=<project_key>` to scope; default = most recent plan.

Method	Path	Purpose
GET	/api/action-plan/progress	All cell values keyed by indicator (<code>"sn\</code>
PUT	/api/action-plan/progress	Single-cell upsert. Body: { indicator, target?, achieved?, notes? }
POST	/api/action-plan/progress/bulk	Bulk upsert. Body: { indicators[], target?, achieved?, notes? }. Max 5000 indicators.
GET	/api/action-plan/history	Recent edits (audit log). <code>?indicator=</code> narrows to one cell
GET	/api/action-plan/export.csv	CSV download of current state

8. WhatsApp Platform

Webhook (public, Meta-signed)

Method	Path
GET	/api/wa/webhook (Meta verification challenge)

Method	Path
POST	/api/wa/webhook (inbound events)

Signature: requires `app_secret` in `wa_config` OR `WA_APP_SECRET` env var. Failures in processing go to `wa_dead_letter`.

Dead-letter queue

Method	Path	Role
GET	/api/wa/dead-letter	admin
POST	/api/wa/dead-letter/:id/retry	admin

Config

Method	Path	Role
GET	/api/wa/config	admin (secrets redacted)
PUT	/api/wa/config	admin

Flows

Method	Path
GET	/api/wa/flows
GET	/api/wa/flows/:id
POST	/api/wa/flows
PUT	/api/wa/flows/:id
DELETE	/api/wa/flows/:id
POST	/api/wa/flows/:id/publish

AI Flow Generator

POST /api/wa/flows/generate

Body: { prompt: string } → returns { name, description, trigger_keywords, nodes } produced by Gemini in JSON mode.

Contacts

Method	Path
GET	/api/wa/contacts?q=&limit=&offset=&opted_in=
GET	/api/wa/contacts/:id/messages
GET	/api/wa/conversations?limit=&q=&assigned=me
POST	/api/wa/contacts/:id/send({ text })
PATCH	/api/wa/contacts/:id/tags({ tags })
POST	/api/wa/contacts/:id/suggest-replies (Gemini returns 3 reply chips)

Collections (tag-based contact groups)

Method	Path
GET	/api/wa/collections
GET	/api/wa/collections/:name/members
POST	/api/wa/collections/:name/add ({ contact_ids: [...] })
POST	/api/wa/collections/:name/remove
DELETE	/api/wa/collections/:name
GET	/api/wa/contact-fields (distinct keys across all contacts.fields JSONB)

Sessions

Method	Path
DELETE	/api/wa/sessions/:id (abandon)
POST	/api/wa/sessions/:id/takeover (agent takes over from bot)
POST	/api/wa/sessions/:id/return-to-bot

Broadcast

POST /api/wa/broadcast

Body: { name, message_body, tags?, recipient_filter? } → { broadcastId, total, status }

Sent in background after immediate ACK. Capped at 1000 recipients.

Analytics

GET /api/wa/analytics

Returns { messageVolume, flowPerformance, nluHealth, broadcasts }. Last 30 days.

Stats / users

Method	Path
GET	/api/wa/stats
GET	/api/wa/assignable-users (managers/admins for handoff assignment)

9. Notebook (RAG over uploaded sources)

All require auth. Most stream SSE (Gemini token-by-token).

Method	Path	Streaming	Purpose
POST	/api/notebook/upload	no	Decode + cap PDF/text source
POST	/api/notebook/fetch-url	no	Pull a web URL as a source
POST	/api/notebook/chat	SSE	RAG chat
POST	/api/notebook/audio	SSE	Audio overview from sources
POST	/api/notebook/tts	no	Text-to-speech audio file
POST	/api/notebook/study-guide	SSE	Study guide PDF
POST	/api/notebook/slide-deck	SSE	PPTX deck
POST	/api/notebook/slide-revise	SSE	Revise an existing slide
POST	/api/notebook/imagen	no	Generate an image

Request body shape (most): { sources: [{ name, content }], message?, history? }

10. Report Writer

Method	Path	Purpose
GET	/api/rw/profile	Org's report-writer profile (tone, language, etc.)
POST	/api/rw/profile/save	Update profile
GET	/api/rw/glossary	Glossary entries
POST	/api/rw/glossary/save	Update glossary
GET	/api/rw/feedback	Feedback history
POST	/api/rw/draft/save	Save a draft
POST	/api/rw/refine/save	Save a refinement
POST	/api/rw/learn	Learn from edits
POST	/api/rw/draft	Generate initial draft (SSE)
POST	/api/rw/refine	Refine an existing draft (SSE)
POST	/api/rw/reflect	Reflect on a draft (SSE)
POST	/api/rw/auto-polish	Auto-polish (SSE)

11. AI memory & settings

POST /api/ai/ask (SSE)

Chat with the org's AI agent. Body: { message, history?, scope? }

Memory CRUD

Method	Path
GET	/api/ai/memories
POST	/api/ai/memories
PUT	/api/ai/memories/:id
DELETE	/api/ai/memories/:id
POST	/api/ai/seed-memories
POST	/api/ai/feedback

Settings & learning (admin)

Method	Path
GET	/api/ai/settings
PUT	/api/ai/settings
GET	/api/ai/health
GET	/api/ai/stats
GET	/api/ai/orgs
GET	/api/ai/learning/candidates
POST	/api/ai/learning/candidates/:id/approve
POST	/api/ai/learning/candidates/:id/reject
POST	/api/ai/learning/rollback
POST	/api/ai/learning/run-batch
GET	/api/ai/rule-versions

12. Prompts (org-level prompt store)

Method	Path	Role
GET	/api/prompts	authed
PUT	/api/prompts/:promptId	admin
DELETE	/api/prompts/:promptId	admin

13. Finance / BigQuery

Method	Path	Purpose
GET	/api/bq/finance	BQ-backed finance dashboard data
GET	/api/bq/attendance	BQ-backed attendance

14. Superadmin

All require role `superadmin`.

Org management

Method	Path
GET	/api/superadmin/orgs
POST	/api/superadmin/orgs
GET	/api/superadmin/stats
PATCH	/api/superadmin/org/:id/metadata
GET	/api/superadmin/org/:id/subscription
POST	/api/superadmin/org/:id/subscription
GET	/api/superadmin/org/:id/users
POST	/api/superadmin/org/:id/users
PATCH	/api/superadmin/org/:id/users/:uid
DELETE	/api/superadmin/org/:id/users/:uid

Plans

Method	Path
GET	/api/superadmin/plans
POST	/api/superadmin/plans
PATCH	/api/superadmin/plans/:id

Billing

Method	Path
GET	/api/superadmin/billing/summary
GET	/api/superadmin/billing/org/:id
POST	/api/superadmin/billing/backfill
GET	/api/superadmin/billing/export.csv

Admin sheet (legacy)

Method	Path
POST	/api/admin/prompt-sheet/create
GET	/api/admin/prompt-sheet

15. Misc

Method	Path	Auth	Purpose
GET	/api/_ah/warmup	public	App Engine warmup (pre-fetches DB)

Method	Path	Auth	Purpose
GET	/api/whatsapp/health	public	Glific legacy health probe
POST	/api/whatsapp/webhook	public	Glific legacy webhook receiver
POST	/api/whatsapp/inbound	public (stub)	Future raw-inbound hook

Conventions

- **All id URL params** for action-plans are constrained to UUID via regex `[0-9a-fA-F-]{36}` so non-UUID paths (e.g. `template.xlsx`) fall through to their own handlers.
- **All cell indicators** are `"sn|location|month"`, e.g. `"3|Khagaria|May"`.
- **Pagination:** `?limit=` (max varies; 60–500) + `?offset=`.
- **Date filters:** ISO format. Most endpoints use a default window (30/90/365 days).
- **SSE responses:** emit `data: {json}\n\n` lines; terminator is `data: [DONE]\n\n`.
- **JSON-mode endpoints:** when Gemini is told to return JSON, parsing errors are caught and surfaced as `{ error: "...", cid }`.

Versioning

There is no explicit `/v1/` prefix. All breaking changes go through a deprecation cycle: old endpoint returns `Warning: 299 - "deprecated, use X"` for at least one release before removal.

Discovery

This file is hand-maintained from grep output (`grep -rEn "router\.(get|post|put|delete|patch)" routes/*.js`). To verify it's current, run that command and diff against the section headers below.